

Assignment Decoded & Selection Strategy

Exactly what the company evaluates, what to build, what to avoid, and the recommended Option 2 execution plan.

Recommended build: Option 2 - ASK -> CLARIFY -> DEFINE -> TEST -> LEARN

Source basis: the complete 13-page assignment PDF provided by you. Page references inside these guides refer to that assignment.

1. What the company actually wants

This assignment is deliberately small. The company is testing whether you can turn an ambiguous natural-language market question into a transparent research process. The strongest submission is not the one with the most features; it is the one that makes assumptions visible, asks the minimum useful clarification questions, structures the experiment cleanly, demonstrates a test, and explains what the result does and does not prove.

Core product idea: Question -> Hypothesis -> Experiment -> Evidence -> Learning. Your UI and architecture should make this transformation visible.

Option 1 vs Option 2

Area	Option 1	Option 2
Scope	Understand -> structure -> clarify -> final experiment	Think note + Ask -> Clarify -> Define -> Test -> Learn
Difficulty	Lower	Moderate
Best for	Fast functional prototype	Strongest selection signal for AI full-stack/product thinking
Deliverables	Prototype, GitHub, README, 2-3 min demo	Prototype, GitHub, Thinking Note, README, AI Usage Note, 2-3 min demo
Recommendation	Good if time is extremely tight	Recommended if you want the best chance to demonstrate reasoning

Evaluation weights - Option 2

Area	Weight	What to demonstrate
Critical Thinking	25%	Spot ambiguity, risk, bias and weak evidence.
Original / Independent Thinking	20%	Make and defend your own product/technical decisions.
Problem Solving	20%	Turn a vague question into an actionable research flow.
Product Thinking	15%	Simple, useful interaction for a normal user.
Technical Implementation	15%	Clean, extensible, working full-stack prototype.
Communication	5%	README, notes and demo are crisp and explainable.

Mandatory behavior your prototype should show

- ASK: accept a natural-language question such as “Does buying NIFTY after a sharp fall work?”
- CLARIFY: detect important missing parameters instead of silently inventing them.
- DEFINE: represent market, condition, entry, exit, holding period, test period, filters, cost assumptions and hypothesis in a structured object.
- TEST: demonstrate how the experiment can be tested using sample, public or simulated data. A production backtester is explicitly not required.
- LEARN: separate “what the data shows” from “what the system concludes”, show a cautious conclusion, and suggest next research questions.

What NOT to build

- No full trading platform.
- No brokerage integration.
- No live order execution.
- No auth unless you genuinely need saved accounts.
- No payment/subscription system.
- No massive dashboard with unrelated widgets.
- No complex agent framework just to look advanced.
- No hidden assumptions.

The biggest failure mode is a beautiful chatbot that directly answers “yes/no”. That would miss the assignment because they want an AI research workflow, not a chat wrapper.

2. Recommended submission concept

Build a single-page “Research Workbench” with five visible stages. The left/top area captures the question; the center shows clarification cards; the final experiment appears as an editable structured panel; “Run sample test” executes deterministic mock/sample analysis; the final section explains evidence, limits and next questions.

Suggested screen flow

Stage	UI	Expected user action
1. Ask	Large research query input + example chips	Enter a question
2. Clarify	2-4 focused missing-field cards with suggestions	Choose/enter values
3. Define	Structured experiment card, editable	Review and confirm
4. Test	Run button + methodology summary + small chart/table	Run sample experiment
5. Learn	Metrics + evidence statement + cautious conclusion + next questions	Understand result / refine experiment

Example interpretation

Field	Value	Source status
Instrument	NIFTY	User said
“Sharp fall”	$\leq -1.0\%$ daily return	Assumption proposed for confirmation
Entry	Buy at next session open after trigger	System asks/confirms
Exit	After 5 trading days	System asks
Holding period	5 sessions	User confirms
Test period	2015-01-01 to 2025-12-31	System asks or offers default
Costs	10 bps round-trip + 5 bps slippage	Visible assumption

Field	Value	Source status
Hypothesis	Post-fall forward return is positive and better than unconditional baseline	System interpretation

Submission checklist

- Deployed prototype link.
- GitHub repository with clean commit history.
- Thinking Note ≤ 2 pages.
- README with architecture, technology choices, key assumptions, run instructions, AI tools, future improvements.
- AI Usage Note ≤ 1 page.
- 2-3 minute complete journey screen recording.

3. Build order that minimizes risk

Order	Build	Done when
1	Static UX shell + state machine	All five stages render with sample state
2	Zod experiment schema	Valid/invalid experiment objects are enforced
3	AI parse endpoint	Question returns structured draft + missing fields
4	Clarification loop	Answers merge into experiment without losing provenance
5	Test engine	Deterministic sample result generated
6	Learn view	Evidence vs conclusion visually separated
7	Security/error states	Rate limit, validation, safe error messages
8	Docs/demo	Everything explainable in 2-3 minutes

Selection strategy: aim for a polished 80% of the core journey rather than 20 unrelated “advanced” features. Every screen should help the evaluator understand your reasoning.

Confirmed Implementation Stack

This page supersedes any earlier wording that presented the AI provider or database as optional choices. For this submission, the selected implementation is fixed:

Layer	Selected technology	How it is used
AI / LLM	Google Gemini API	Parse natural-language research questions, identify ambiguity, propose clarification questions
Database	Neon PostgreSQL	Persist research sessions, structured experiments, clarification answers, test runs, and confirmation
ORM	Drizzle ORM	Typed schema, migrations, and database access from the Next.js server layer.
Validation	Zod	Validate both user input and Gemini structured output before persistence or testing.
Frontend / Backend	Next.js + TypeScript	App Router UI plus server-side route handlers/server functions.

Environment variables

```
GEMINI_API_KEY=...
DATABASE_URL=postgresql://...
GEMINI_MODEL=gemini-2.5-flash
```

Recommended data flow

Question -> server validation -> Gemini structured interpretation -> Zod validation -> clarification/confirmation -> experiment persisted to Neon -> deterministic TypeScript test -> result persisted -> Learn view.

Security rules

Never expose GEMINI_API_KEY or DATABASE_URL in client code. Keep both outside NEXT_PUBLIC variables. Validate request sizes, parse Gemini output against a strict schema, use parameterized ORM queries, add API rate limiting, and never let the LLM fabricate backtest metrics.

Suggested Neon schema

research_sessions: id, original_question, status, created_at
experiments: id, session_id, instrument, timeframe, entry_condition, exit_condition, holding_period, test_period, filters_json, costs_json, hypothesis, provenance_json
test_runs: id, experiment_id, dataset_source, metrics_json, warnings_json, created_at

Why this choice is strong for the assignment: Gemini is used for language understanding and ambiguity handling, while Neon stores reproducible research state. Numerical evidence remains deterministic and auditable instead of being invented by the model.